

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА №3-4

«Функциональные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкция языков программирования»

Выполнил: студент гр. ИУ5-34Б _____ (Бондаренко Д.А.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

1. Описание задания

Цель работы: изучение возможностей функционального программирования в языке Python.

В соответствии с заданием лабораторной работы №3-4 необходимо разработать набор функций, генераторов, классов-итераторов, декораторов и контекстных менеджеров, реализующих парадигмы функционального программирования в языке Python 3. Все исходные файлы программы должны быть организованы в рамках пакета lab_python_fp.

В рамках лабораторной работы решаются следующие задачи:

- Задача 1 (файл field.py): разработка генератора field, последовательно извлекающего значения заданных ключей из списка словарей. Поддерживается обработка как одного ключа (выдача значений элементов без None), так и нескольких ключей (формирование отфильтрованных словарей).
- Задача 2 (файл gen_random.py): разработка генератора gen_random(num_count, begin, end), возвращающего заданное количество псевдослучайных целых чисел в диапазоне от минимума до максимума включительно.
- Задача 3 (файл unique.py): разработка класса-итератора Unique(items, **kwargs), отфильтровывающего дубликаты из переданного списка или генератора с опциональным игнорированием регистра строковых значений (параметр ignore_case).
- Задача 4 (файл sort.py): сортировка одномерного числового массива по модулю в порядке убывания в одну строку кода двумя способами — с использованием lambda-выражения и без него.
- Задача 5 (файл print_result.py): разработка функции-декоратора @print_result, выполняющей логирование имени вызываемой функции и ее результата со специальным построчным форматированием для списков (построчно) и словарей (в формате «ключ = значение»).
- Задача 6 (файл cm_timer.py): реализация контекстных менеджеров cm_timer_1 и cm_timer_2 для хронометража выполнения блоков инструкций с использованием класса (с методами __enter__ и __exit__) и библиотеки contextlib (декоратор @contextmanager).
- Задача 7 (файл process_data.py): построение сквозного конвейера функциональной обработки данных реального датасета вакансий

(data_light.json) с использованием функций f1, f2, f3, f4, декоратора @print_result и контекстного таймера cm_timer_1.

2. Текст программы

2.1. Генератор field — файл lab_python_fp/field.py

```
def field(items, *args):
    assert len(args) > 0, "Необходимо указать хотя бы одно поле"

    if len(args) == 1:
        key = args[0]
        for item in items:
            val = item.get(key)
            if val is not None:
                yield val
    else:
        for item in items:
            filtered = {k: item[k] for k in args if k in item and item[k] is not None}
            if len(filtered) > 0:
                yield filtered


if __name__ == '__main__':
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'},
        {'color': 'blue'},
        {'title': None, 'price': None}
    ]

    print("Один аргумент ('title'):")
    print(list(field(goods, 'title')))

    print("\nНесколько аргументов ('title', 'price'):")
    print(list(field(goods, 'title', 'price')))
```

2.2. Генератор gen_random — файл lab_python_fp/gen_random.py

```
import random
```

```
def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)


if __name__ == '__main__':
    print(list(gen_random(5, 1, 3)))
    print(list(gen_random(10, 10, 20)))
```

2.3. Итератор уникальных элементов — файл lab_python_fp/unique.py

```
class Unique(object):
    def __init__(self, items, **kwargs):
        self.ignore_case = kwargs.get('ignore_case', False)
        self.items_iter = iter(items)
        self.used_elements = set()

    def __iter__(self):
        return self

    def __next__(self):
        while True:
            item = next(self.items_iter)

            # Ключ для проверки дубликатов в хэш-таблице
            if self.ignore_case and isinstance(item, str):
                check_item = item.lower()
            else:
                check_item = item

            if check_item not in self.used_elements:
                self.used_elements.add(check_item)
                return item


if __name__ == '__main__':
    from gen_random import gen_random
```

```
    data1 = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
    print("data1:", list(Unique(data1)))

    data2 = gen_random(10, 1, 3)
    print("data2 (генератор):", list(Unique(data2)))

    data3 = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
    print("data3 (ignore_case=False):", list(Unique(data3)))
    print("data3 (ignore_case=True):", list(Unique(data3, ignore_case=True)))
```

2.4. Сортировка по модулю — файл lab_python_fp/sort.py

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    # 1. Без использования lambda-функции
    result = sorted(data, key=abs, reverse=True)
    print(result)

    # 2. С использованием lambda-функции
    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print(result_with_lambda)
```

2.5. Декоратор print_result — файл lab_python_fp/print_result.py

```
from functools import wraps


def print_result(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(func.__name__)
        result = func(*args, **kwargs)
        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
```

```
            for k, v in result.items():
                print(f"{k} = {v}")
        else:
            print(result)
        return result
    return wrapper


@print_result
def test_1():
    return 1


@print_result
def test_2():
    return 'iu5'


@print_result
def test_3():
    return {'a': 1, 'b': 2}


@print_result
def test_4():
    return [1, 2]


if __name__ == '__main__':
    print('!!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()
```

2.6. Контекстные менеджеры времени — файл lab_python_fp/cm_timer.py

```
import time
from contextlib import contextmanager
```

```
class cm_timer_1:
    def __enter__(self):
        self.start_time = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.perf_counter() - self.start_time
        print(f"time: {elapsed:.2f}")


@contextmanager
def cm_timer_2():
    start_time = time.perf_counter()
    try:
        yield
    finally:
        elapsed = time.perf_counter() - start_time
        print(f"time: {elapsed:.2f}")


if __name__ == '__main__':
    print("Тестирование cm_timer_1:")
    with cm_timer_1():
        time.sleep(1.5)

    print("\nТестирование cm_timer_2:")
    with cm_timer_2():
        time.sleep(1.5)
```

2.7. Обработка данных вакансий — файл lab_python_fp/process_data.py

```
import json
import sys
import os

from print_result import print_result
from cm_timer import cm_timer_1
```

```
from unique import Unique
from field import field
from gen_random import gen_random

# Чтение аргумента командной строки или использование файла по
умолчанию
path = sys.argv[1] if len(sys.argv) > 1 else 'data_light.json'

if not os.path.exists(path):
    alt_path = os.path.join(os.path.dirname(__file__), path)
    if os.path.exists(alt_path):
        path = alt_path

with open(path, encoding='utf-8') as f:
    data = json.load(f)


@print_result
def f1(arg):
    return sorted(list(Unique(field(arg, 'job-name'), ignore_case=True)),
key=lambda s: s.lower())


@print_result
def f2(arg):
    return list(filter(lambda s: s.lower().startswith('программист'), arg))


@print_result
def f3(arg):
    return list(map(lambda s: f"{s} с опытом Python", arg))

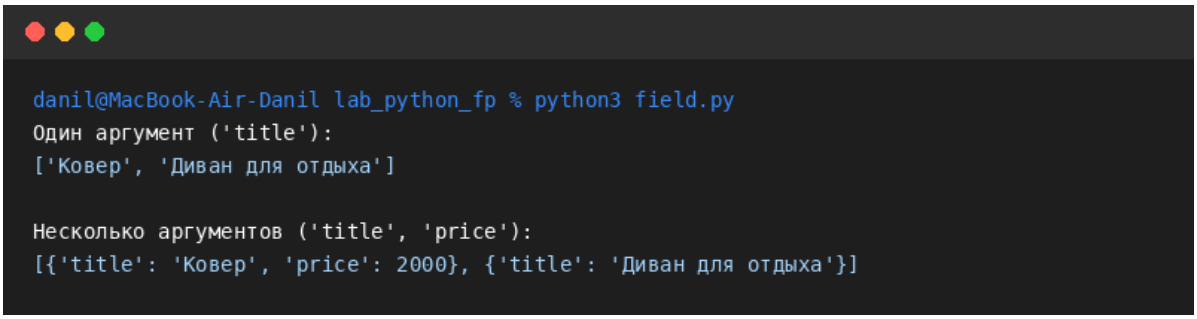

@print_result
def f4(arg):
    salaries = gen_random(len(arg), 100000, 200000)
    return [f"{job}, зарплата {salary} руб." for job, salary in zip(arg, salaries)]


if __name__ == '__main__':
```

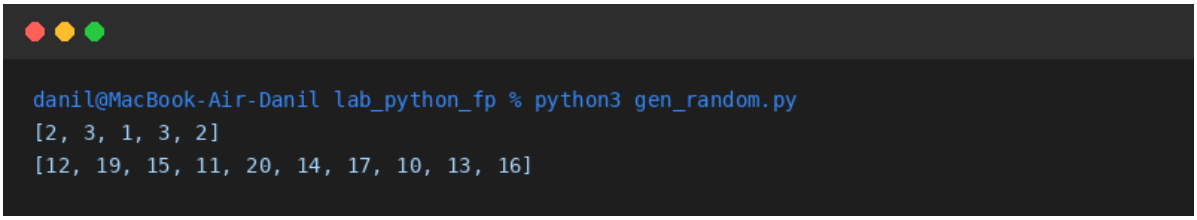
```
with cm_timer_1():
    f4(f3(f2(f1(data))))
```

3. Экранные формы с примерами выполнения программы

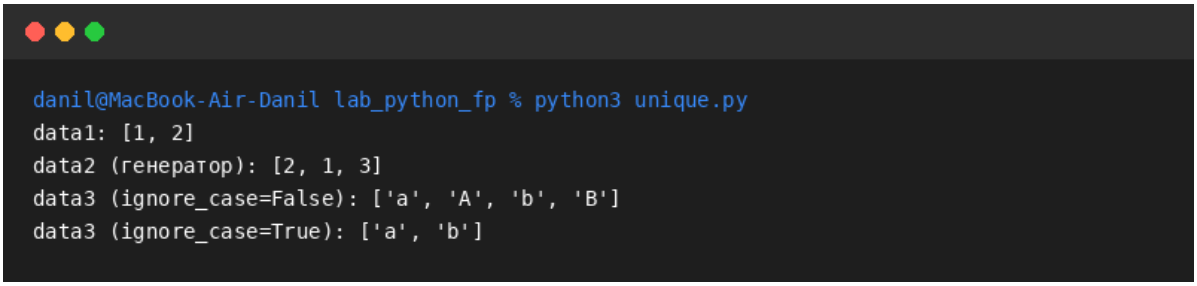
3.1. Запуск генератора field (field.py)



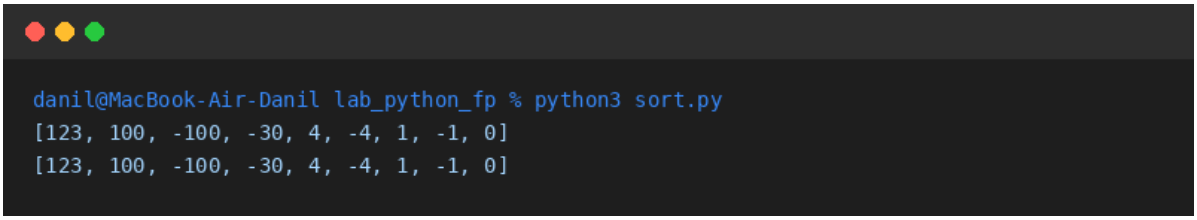
3.2. Запуск генератора случайных чисел (gen_random.py)



3.3. Запуск итератора уникальных значений (unique.py)



3.4. Запуск сортировки по модулю (sort.py)



3.5. Запуск декоратора вывода результатов (print_result.py)

```
danil@MacBook-Air-Danil lab_python_fp % python3 print_result.py
!!!!!!!!
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2
```

3.6. Запуск контекстных таймеров (cm_timer.py)

```
danil@MacBook-Air-Danil lab_python_fp % python3 cm_timer.py
Тестирование cm_timer_1:
time: 1.50

Тестирование cm_timer_2:
time: 1.50
```

3.7. Запуск конвейера обработки данных (process_data.py)

```
danil@MacBook-Air-Danil lr3-4 % cd lab_python_fp
danil@MacBook-Air-Danil lab_python_fp % python3 process_data.py ../data_light.json
f1
Аналитик данных
Дизайнер интерфейсов
Программист C#
программист Java
Программист Python
f2
Программист C#
программист Java
Программист Python
f3
Программист C# с опытом Python
программист Java с опытом Python
Программист Python с опытом Python
f4
Программист C# с опытом Python, зарплата 177274 руб.
программист Java с опытом Python, зарплата 184148 руб.
Программист Python с опытом Python, зарплата 145013 руб.
time: 0.00
```

Вывод

В ходе выполнения лабораторной работы №3-4 были изучены и практически освоены возможности функционального программирования в языке Python 3, а также разработана модульная программа в пакете lab_python_fp для конвейерной обработки структурированных данных.

В процессе выполнения работы были приобретены следующие ключевые навыки:

- реализация пользовательских функций-генераторов на основе ключевого слова yield (генераторы field и gen_random), обеспечивающих ленивые (отложенные) вычисления и оптимизацию расхода оперативной памяти при обработке коллекций;
- проектирование классов-итераторов с реализацией протокола итератора Python (методы __iter__ и __next__), поддержка обработки потоков произвольных итерируемых типов (списки, кортежи, генераторы) и фильтрация повторяющихся значений с помощью хэш-таблиц (set) и опционального игнорирования регистра;

- применение функций высшего порядка (sorted, filter, map) и анонимных функций (lambda-выражений) для компактной декларативной обработки коллекций;
- разработка пользовательских декораторов (декоратор @print_result с сохранением метаданных через functools.wraps), позволяющих расширять поведение функций без изменения их исходного кода;
- создание менеджеров контекста двумя ключевыми способами: через реализацию класса с протоколом __enter__ / __exit__ и декларативным методом с применением генератора и декоратора @contextmanager модуля contextlib;
- построение полного конвейера функционального преобразования реальных JSON-данных через композицию функций f1 → f2 → f3 → f4, использование функции zip для параллельной агрегации потоков и замер производительности через собственный контекстный таймер.